

گزارش جامع پروژه

برنامه‌ریزی و بهینه‌سازی تولید تویوتا

سامانه هوشمند مبتنی بر برنامه‌ریزی MILP، الگوریتم ژنتیک و هوش مصنوعی

نام دانشجو: آرام شفیع‌ها

استاد درس: دکتر قدوسی

درس: برنامه‌نویسی پیشرفته

مخزن GitHub پروژه

https://github.com/aramshafieha/Toyota_Production_Planning

گزارش آکادمیک پروژه — ۱۴۰۵

سامانه هوشمند مبتنی بر برنامه‌ریزی MILP، الگوریتم ژنتیک و هوش مصنوعی

گزارش جامع پروژه

برنامه‌ریزی و بهینه‌سازی تولید تویوتا

نام دانشجو: آرام شفیع‌ها

استاد درس: دکتر قدوسی

درس: برنامه‌نویسی پیشرفته

دانشکده مهندسی صنایع

درس برنامه‌نویسی پیشرفته

گزارش جامع آکادمیک پروژه

سامانه برنامه‌ریزی و بهینه‌سازی تولید تویوتا

Toyota Production Planning Decision Support System

مشخصات	مقدار
نوع پروژه	اپلیکیشن تصمیم‌گیری مهندسی صنایع مبتنی بر Python
معماری	چهار ماژول: LP/MILP، الگوریتم ژنتیک، رابط وب، دستیار AI
رابط کاربری	Streamlit
حل‌گر دقیق	PuLP
الگوریتم فراابتکاری	Genetic Algorithm
دستیار هوشمند	Google Gemini API
نسخه گزارش	نسخه نهایی مستندسازی
آرام شفیع‌ها	
دکتر قدوسی	

چکیده

این پروژه یک سامانه تصمیم‌یار برای برنامه‌ریزی تولید در یک کارخانه خودروسازی است که با ترکیب بهینه‌سازی دقیق، الگوریتم‌های تکاملی و هوش مصنوعی مولد طراحی شده است. هسته دقیق سامانه، یک مدل برنامه‌ریزی خطی عدد صحیح مختلط (MILP) است که با کتابخانه PuLP پیاده‌سازی شده و هدف آن بیشینه‌سازی سود تولید تحت محدودیت‌های ظرفیت، منابع و تقاضای بازار است. در کنار آن، یک الگوریتم ژنتیک برای حل مسئله ترتیبی و زمان‌بندی تولید طراحی شده است؛ در این بخش، توالی مدل‌های خودرو به‌عنوان کروموزوم، تابع Fitness به‌عنوان معیار کیفیت و عملگرهای Crossover و Mutation به‌عنوان سازوکار جست‌وجوی فضای جواب استفاده می‌شوند.

رابط کاربری با Streamlit توسعه یافته و امکان فراخوانی ماژول‌های محاسباتی، مشاهده خروجی‌های جدولی و نموداری و تعامل با دستیار هوشمند Gemini را فراهم می‌کند. همچنین، پارامترهای الگوریتم ژنتیک مانند اندازه جمعیت و تعداد نسل‌ها و پارامترهای ظرفیت تولید در نسخه تعاملی داشبورد قابل تغییر هستند تا کاربر بتواند سناریوهای مختلف را آزمایش کند. ساختار پروژه با الزامات پروژه درس برنامه‌نویسی پیشرفته، شامل چهار جزء حل‌کننده خطی، الگوریتم ژنتیک، Application Web و Chatbot، هم‌راستا شده است.

در طراحی گزارش حاضر تلاش شده است ضمن ارائه فرمول‌بندی ریاضی، منطق توابع کلیدی، جریان داده، معماری نرم‌افزار، نحوه تعامل ماژول‌ها، مزایا، محدودیت‌ها و مسیرهای توسعه آتی به صورت منظم و قابل ارائه مستند شوند.

کلیدواژه‌ها

برنامه‌ریزی تولید، Gemini، Streamlit، Mutation، Crossover، Genetic Algorithm، PuLP، MILP، بهینه‌سازی، سیستم تصمیم‌یار

فهرست مطالب

۱. مقدمه و تعریف مسئله

۲. انطباق پروژه با الزامات درس
۳. معماری و طراحی نرم افزار
۴. داده ها و پارامترهای مسئله
۵. مدل برنامه ریزی خطی عدد صحیح مختلط (MILP)
۶. تحلیل Slack و Shadow Price
۷. الگوریتم ژنتیک و مسئله NP-Hard
۸. تابع Crossover، Fitness، و Mutation
۹. رابط کاربری Streamlit
۱۰. دستیار هوشمند Gemini
۱۱. جریان اجرای سامانه و مدیریت خطا
۱۲. مستندسازی توابع و ماژول ها
۱۳. آزمون و ارزیابی
۱۴. محدودیت ها و پیشنهادهای توسعه
۱۵. جمع بندی
۱۶. پیوست: ساختار فایل ها و چک لیست تحویل

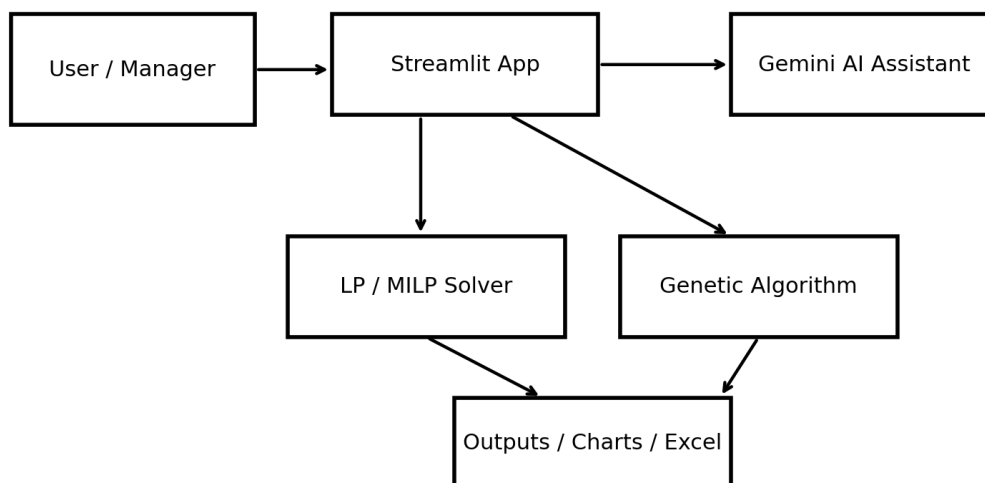
۱. مقدمه و تعریف مسئله

مسئله تخصیص ترکیب تولید در کارخانه خودروسازی از جنس مسائل تصمیم گیری چندمحدودیته است: برای هر مدل خودرو باید تعداد تولیدی تعیین شود، در حالی که ظرفیت بخش هایی مانند موتور، انتقال قدرت، رنگ، بدنه و مونتاژ محدود است و سقف تقاضای بازار نیز باید رعایت شود. دستورالعمل پروژه نیز برای سیستم «برنامه ریزی تولید کارخانه خودروسازی» تعیین ترکیب بهینه تولید محصولات با محدودیت قطعات و ظرفیت سالن های مونتاژ را به عنوان مسئله خطی MILP/LP و زمان بندی کارگاهی و ماشینکاری قطعات موتوری را به عنوان مسئله NP-Hard مبتنی بر GA معرفی می کند. ❧

در این پروژه، بخش اول به صورت یک مدل MILP با متغیرهای تصمیم صحیح پیاده سازی شده است. مدل با هدف بیشینه سازی سود کل، میزان تولید پنج مدل Prius و Corolla، Camry، RAV4، Highlander را

تعیین می‌کند. داده‌های سود، زمان/مصرف منابع و تقاضا از فایل CSV خوانده می‌شوند. در کد موجود، متغیرهای تصمیم هر پنج مدل با نوع Integer تعریف شده‌اند و مدل در حالت Maximize ساخته می‌شود.

Modular Architecture of the Toyota Production Planning System



شکل 1 — معماری ماژولار سامانه تصمیم‌یار تولید

1-1. اهداف پروژه

- ایجاد یک مدل دقیق برای ترکیب بهینه تولید و بیشینه‌سازی سود.
- پیاده‌سازی یک الگوریتم ژنتیک سفارشی برای جست‌وجوی توالی‌های تولید.
- ارائه داشبورد وب برای فراخوانی ماژول‌ها و مشاهده نتایج.
- فراهم کردن تعامل زنده با کاربر از طریق دستیار هوشمند Gemini.
- مستندسازی ریاضی، نرم‌افزاری و تجربی پروژه به صورت یکپارچه.

۲. انطباق پروژه با الزامات درس

دستورالعمل پروژه بر معماری ماژولار چهارگانه شامل حل‌کننده خطی، الگوریتم ژنتیک مبتنی بر AI، رابط کاربری وب و دستیار چت‌بات تأکید دارد. همچنین، در بخش گزارش نهایی، نگارش حرفه‌ای، گرافیک، نمودارهای ریاضی و مستندسازی دقیق توابع مورد انتظار است.

وضعیت	پیاده‌سازی پروژه	بخش مورد انتظار
✓	تعریف متغیرهای تولید، تابع هدف و قیود ظرفیت/تقاضا	مدل‌سازی مهندسی صنایع
✓	+ PuLP + LpProblem متغیرهای Integer	LP/MILP Solver
*✓	استخراج constraint.pi و constraint.slack پس از حل	Shadow Price / Slack
✓	،Population، Fitness ،Selection، Crossover Mutation، Generations	Genetic Algorithm
✓	Streamlit با تب‌های LP، GA و AI	Interactive App
✓	ظرفیت‌های LP و Population /Generations در نسخه تعاملی	پارامترهای قابل تغییر
✓	نمودار تولید و نمودارهای تحلیلی گزارش	تحلیل نموداری
✓	پرسش‌های تولید و بهینه‌سازی برای Google Gemini API	Chatbot

* نکته: چون مدل اصلی به صورت MILP و متغیرهای تصمیم Integer تعریف شده است، مقدار Shadow Price استخراج‌شده از حل‌کننده CBC لزوماً مانند یک LP پیوسته تفسیر اقتصادی مستقیم ندارد. برای تحلیل حساسیت کلاسیک، حل نسخه LP پیوسته مناسب‌تر است.

2-1. معماری چهارماژوله

1. ماژول حل دقیق: مدل MILP تولید و محاسبه جواب بهینه.
2. ماژول Genetic Algorithm: بهینه‌سازی توالی و زمان‌بندی تولید.
3. ماژول Application: داشبورد Streamlit و نمایش خروجی.

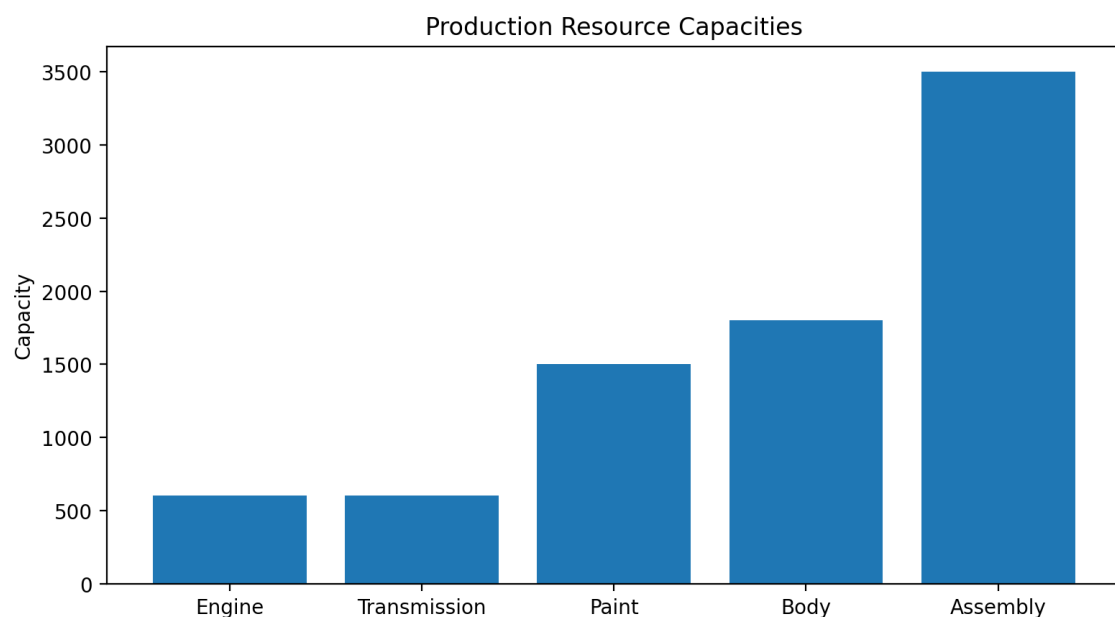
4. ماژول Chatbot: پاسخ‌گویی تحلیلی مبتنی بر Gemini.

۳. داده‌ها و پارامترهای مسئله

ماژول LP داده‌های ورودی را از فایل production_data.csv دریافت می‌کند. در کد، ستون‌های Vehicle، Profit، Paint، Body، Assembly و Demand به دیکشنری‌های پارامتری تبدیل می‌شوند. این ساختار باعث می‌شود منطق مدل از داده‌های ورودی جدا باشد و تغییر داده‌ها نیازمند بازنویسی فرمول‌بندی اصلی نباشد.

پارامتر	معنی	کاربرد
Vehicle	نام مدل خودرو	اندیس متغیر تصمیم
Profit	سود هر واحد	ضریب تابع هدف
Paint	مصرف ظرفیت رنگ	قید ظرفیت رنگ
Body	مصرف ظرفیت بدنه	قید بدنه
Assembly	مصرف ظرفیت مونتاژ	قید مونتاژ
Demand	حداکثر تقاضای بازار	قیود تقاضا

3-1. ظرفیت‌های نمونه تعریف‌شده در مدل



شکل 2 — ظرفیت‌های تعریف‌شده برای منابع اصلی مدل LP

در نسخه پایه کد، ظرفیت موتور و انتقال قدرت برابر 600 واحد، ظرفیت رنگ 1500، ظرفیت بدنه 1800 و ظرفیت مونتاژ 3500 واحد/واحدزمان تعریف شده است. در نسخه تعاملی توسعه‌یافته، این ظرفیت‌ها از رابط کاربر قابل تنظیم شده‌اند و سپس به حل‌کننده ارسال می‌شوند.

۴. مدل برنامه‌ریزی خطی عدد صحیح مختلط (MILP)

متغیر تصمیم x_i تعداد تولید هر مدل خودرو در افق برنامه‌ریزی است. از آنجا که تولید خودرو به واحدهای صحیح نیاز دارد، متغیرها در کد با "cat=Integer" تعریف شده‌اند؛ بنابراین مدل از نوع MILP/Integer Linear Programming است.

4-1. متغیرهای تصمیم

برای پنج مدل خودرو داریم:

نماد	مدل
x_1	Corolla
x_2	Camry
x_3	RAV4
x_4	Highlander
x_5	Prius

4-2. تابع هدف

$$\max Z = \sum_{i \in V} Profit_i x_i$$

شکل 3 — تابع هدف بیشینه‌سازی سود کل

در کد، ضرایب تابع هدف از ستون Profit فایل داده خوانده شده و به صورت مجموع سود واحد هر مدل ضربدر تعداد تولید آن مدل ساخته می‌شوند. هدف مدل، بیشینه‌سازی سود کل تولید است.

4-3. قیود ظرفیت

$$\sum_i x_i \leq 600, \quad \sum_i Paint_i x_i \leq 1500, \quad \sum_i Body_i x_i \leq 1800, \quad \sum_i Assembly_i x_i \leq 3500$$

شکل 4 — فرم فشرده قیود ظرفیت اصلی

قید ظرفیت موتور و انتقال قدرت به صورت مجموع تعداد تولید و با سقف 600 تعریف شده است. قیود رنگ، بدنه و مونتاژ نیز با ضرایب مصرف هر مدل تعریف شده‌اند. علاوه بر این، برای هر خودرو قید $x_i \geq Demand_i$ برقرار است.

4-4. حل مدل

پس از ساخت مدل، دستور `model.solve()` فراخوانی می‌شود. وضعیت حل، مقادیر بهینه متغیرها و مقدار تابع هدف از طریق `LpStatus`، `value` و `model.objective` استخراج می‌شوند. خروجی تولید در `production_plan.xlsx` ذخیره و نمودار تولید در `production_chart.png` تولید می‌شود.

۵. تحلیل Slack و Shadow Price

پس از حل مدل، پروژه برای هر `constraint` مقدار `constraint.pi` و `constraint.slack` را استخراج می‌کند. Slack نشان می‌دهد چه مقدار ظرفیت/فضای باقیمانده در یک قید وجود دارد. Shadow Price در یک LP پیوسته بیانگر تغییر حاشیه‌ای مقدار تابع هدف در اثر افزایش یک واحد سمت راست قید، در محدوده اعتبار تحلیل حساسیت، است.

شاخص	تفسیر
Slack = 0	قید فعال/Binding است و تمام ظرفیت مؤثر مصرف شده است.
Slack > 0	بخشی از ظرفیت یا سهم قید بلااستفاده مانده است.
Shadow Price $\neq$ 0	در LP پیوسته، افزایش RHS قید می‌تواند ارزش هدف را تغییر دهد.
Shadow Price = 0	قید در تحلیل دوگان LP پیوسته ارزش حاشیه‌ای صفر دارد؛ در MILP نیز π را نباید بدون احتیاط تفسیر کرد.

در نسخه فعلی مدل، چون متغیرهای تصمیم Integer هستند، صفر بودن Shadow Price ها می‌تواند ناشی از محدودیت تفسیر دوگان در حل MILP باشد. بنابراین برای ارائه علمی، بهتر است Shadow Price به عنوان «خروجی تحلیل حساسیت حل‌کننده» گزارش شود و برای تفسیر اقتصادی کلاسیک، یک Relaxation پیوسته از مدل نیز در نظر گرفته شود.

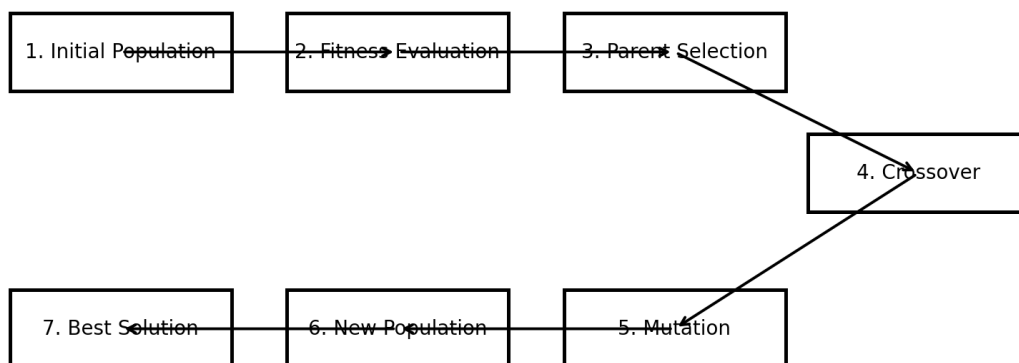
5-1. نمایش در داشبورد

در طراحی نهایی داشبورد، جدول Shadow Price/Slack باید درون تب Linear Programming و پس از اجرای حل‌کننده نمایش داده شود؛ به این ترتیب خروجی تحلیل حساسیت با نتیجه همان اجرای مدل همزمان خواهد بود.

6. الگوریتم ژنتیک و مسئله NP-Hard

دومین بخش محاسباتی پروژه به زمان‌بندی/توالی تولید اختصاص دارد. فضای جواب‌های ترتیبی با افزایش تعداد عملیات و محدودیت‌ها به سرعت بزرگ می‌شود و در صورت توسعه به Job-Shop Scheduling، ماهیت NP-Hard پیدا می‌کند. در نسخه پیاده‌سازی شده، کروموزوم یک permutation از پنج مدل خودرو است.

Genetic Algorithm Workflow

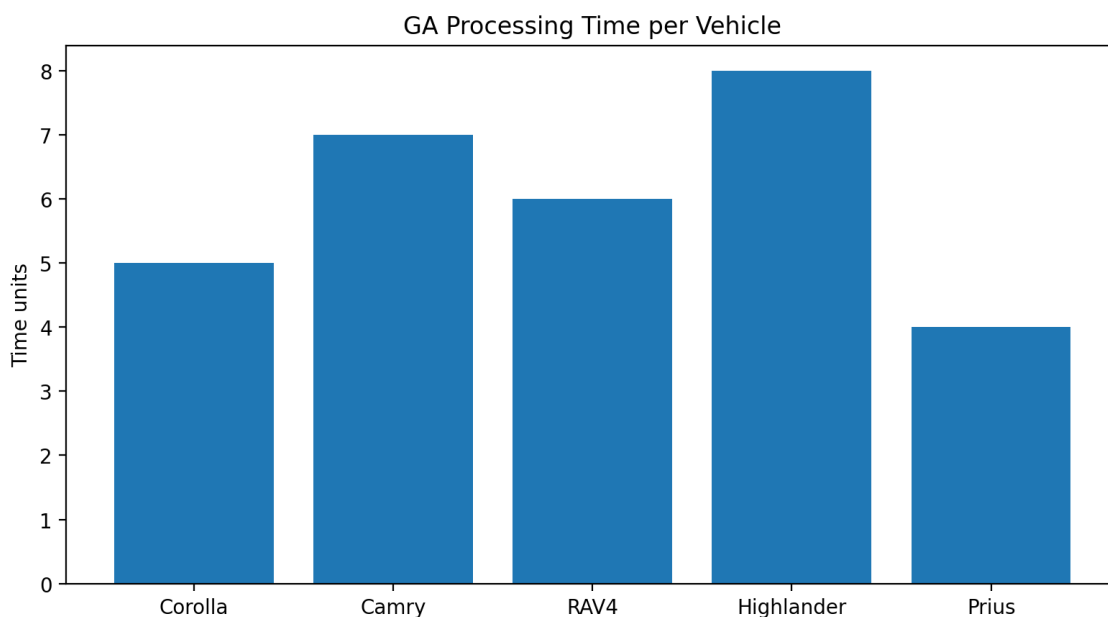


شکل 5 — جریان اصلی الگوریتم ژنتیک

6-1. نمایش کروموزوم

هر کروموزوم یک ترتیب از خودروهاست؛ برای مثال [Corolla, Camry, Prius, RAV4, Highlander]. استفاده از random.sample تضمین می‌کند که هر خودرو دقیقاً یک بار در کروموزوم حضور داشته باشد.

6-2. زمان پردازش



شکل 6 — زمان پردازش تعریف‌شده برای مدل‌های خودرو

زمان‌های پردازش تعریف شده در نسخه توسعه یافته به ترتیب $Corolla=5$ ، $Camry=7$ ، $RAV4=6$ ، $Prius=4$ و $Highlander=8$ هستند. مجموع این زمان‌ها به عنوان makespan ساده شده در ارزیابی توالی مورد استفاده قرار می‌گیرد.

۷. تابع Fitness، Crossover و Mutation

7-1. تابع Fitness

$$Fitness(c) = 100 - 5\max(0, T(c) - C) + P(c) + \max(0, 50 - T(c))$$

شکل 7 — فرم فشرده تابع Fitness

تابع Fitness از یک امتیاز پایه 100 شروع می‌کند. اگر زمان کل از ظرفیت مونتاژ بیشتر شود، به ازای مازاد زمان جریمه اعمال می‌شود. سپس برای قرارگیری Corolla در جایگاه اول، Camry در جایگاه دوم و Prius در جایگاه سوم امتیازهای ترجیحی اضافه می‌شود. در نهایت، توالی‌های کوتاه‌تر امتیاز بیشتری از بخش makespan دریافت می‌کنند.

این تابع در واقع یک Fitness سفارشی و ترکیبی است که هم محدودیت ظرفیت را در نظر می‌گیرد و هم ترجیحات توالی را. چنین طراحی‌ای برای پروژه آموزشی مناسب است، زیرا منطق الگوریتم و نحوه تعریف معیار کیفیت را به صورت شفاف نشان می‌دهد.

7-2. Selection

در هر نسل، جمعیت بر اساس Fitness به صورت نزولی مرتب می‌شود و دو عضو اول به عنوان والد انتخاب می‌شوند. این روش ساده و قابل توضیح است و نوعی انتخاب حریصانه مبتنی بر بهترین افراد ایجاد می‌کند.

7-3. Crossover

برای حفظ ساختار permutation، یک نقطه برش تصادفی انتخاب می‌شود. بخش ابتدایی از parent1 برداشته شده و ژن‌های موجود در parent2 که هنوز در child حضور ندارند به ترتیب اضافه می‌شوند. این روش از ایجاد ژن تکراری جلوگیری می‌کند.

Mutation .7-4

با احتمال 20 درصد، دو موقعیت به صورت تصادفی انتخاب و جای آن‌ها عوض می‌شود. این Swap Mutation تنوع جمعیت را حفظ می‌کند و مانع همگرایی زودرس به یک ترتیب خاص می‌شود.

۸. رابط کاربری Streamlit

رابط وب با Streamlit ساخته شده است. کد app.py صفحه را با عنوان Toyota Production Planning و سه تب اصلی Linear Programming، Genetic Algorithm و AI Assistant سازمان‌دهی می‌کند. در Sidebar وضعیت سیستم و ماژول‌های فعال نمایش داده می‌شود و در Header نیز هدف داشبورد و فناوری‌های به‌کاررفته معرفی شده‌اند.

بخش	عملکرد
Sidebar	لوگو، وضعیت سیستم و معرفی ماژول‌ها
Status Cards	نمایش وضعیت مدل‌ها، GA، Optimization و AI
LP Tab	اجرای حل‌کننده، جدول تولید و نمودار
GA Tab	تنظیم Population/Generations و اجرای GA
AI Tab	ورود سؤال و دریافت پاسخ Gemini

8-1. طراحی ظاهری

برای ایجاد ظاهر حرفه‌ای، CSS سفارشی در app.py استفاده شده است: پس‌زمینه روشن، دکمه‌های قرمز با گوشه‌های گرد، کارت‌های Metric با سایه، Sidebar تیره و Header گردانی. این انتخاب‌ها با هویت بصری Toyota و هدف ارائه دانشگاهی هماهنگ هستند.

8-2. تعامل‌پذیری پارامترها

برای برآورده کردن الزام «قابلیت تغییر مشخصات و پارامترهای مسئله»، ورودی‌های عددی/کشویی برای ظرفیت‌های LP و پارامترهای GA مانند Population Size و Generations در نسخه توسعه‌یافته داشبورد قرار گرفته‌اند. این مقادیر از طریق subprocess به ماژول‌های محاسباتی ارسال می‌شوند.

۹. دستیار هوشمند Gemini

ماژول Chatbot از Google Gemini API استفاده می‌کند. کلید API از فایل env. با load_dotenv خوانده شده و با genai.Client به سرویس متصل می‌شود. سپس متن پرسش کاربر در قالب یک prompt شامل دامنه‌های Linear Programming, Genetic Algorithm, Toyota Production System و Manufacturing Optimization به مدل ارسال می‌شود.

این بخش یک لایه تعاملی بر روی خروجی‌های مهندسی ایجاد می‌کند: کاربر می‌تواند درباره مفاهیم برنامه‌ریزی تولید، LP، GA و سیستم تولید سؤال بپرسد و پاسخ متنی دریافت کند. از منظر معماری، Chatbot یک مصرف‌کننده مستقل از نتایج محاسباتی است و در نسخه‌های بعدی می‌تواند مستقیماً فایل خروجی و وضعیت اجرای آخرین سناریو را نیز به context مدل اضافه کند.

9-1. ملاحظات امنیتی

کلید API نباید در کد، گزارش عمومی یا مخزن GitHub قرار گیرد و باید از env یا Secret Manager استفاده شود. در فایل‌های تصویری محیط توسعه نیز نباید کلید API قابل مشاهده باشد. این اصل برای تحویل نهایی و انتشار کد ضروری است.

۱۰. جریان اجرای سامانه و مدیریت خطا

در زمان اجرای LP، داشبورد ماژول lp_solver.py را فراخوانی می‌کند. پس از پایان حل، فایل Excel و نمودار تولید در پوشه outputs قرار می‌گیرند و Streamlit آن‌ها را خوانده و نمایش می‌دهد. در اجرای GA نیز خروجی متنی الگوریتم از طریق subprocess دریافت می‌شود.

10-1. جریان داده

1. کاربر پارامترهای مسئله را در داشبورد وارد می‌کند.
2. app.py پارامترها را به ماژول محاسباتی ارسال می‌کند.
3. lp_solver.py یا ga_scheduler.py مسئله را اجرا می‌کند.
4. نتیجه به صورت متن، جدول، فایل Excel یا تصویر نمودار تولید می‌شود.
5. Streamlit خروجی را به کاربر نمایش می‌دهد.
6. در صورت نیاز، Chatbot توضیح مفهومی و تحلیلی ارائه می‌کند.

2-10. مدیریت خطا

برای فرآیندهای subprocess باید returncode و stderr بررسی شوند تا در صورت شکست اجرای ماژول، به جای نمایش موفقیت کاذب، پیام خطا به کاربر داده شود. همچنین وجود فایل‌های خروجی مانند production_plan.xlsx و production_chart.png باید قبل از خواندن کنترل شود. این الگو از بروز خطاهای ثانویه در داشبورد جلوگیری می‌کند.

۱۱. مستندسازی توابع و ماژول‌ها

ماژول/تابع	ورودی	خروجی	نقش
app.py / Streamlit UI	پارامترهای کاربر و پرسش	نمایش وب	Orchestration و GUI
lp_solver.py	CSV + ظرفیت‌ها	جواب بهینه، /objective، Slack ،Shadow Price Excel، chart	حل MILP
fitness(chromosome)	لیست permutation	Fitness عددی	ارزیابی کیفیت کروموزوم
Initial Population	vehicles، population_size	لیست کروموزوم‌ها	تولید جمعیت اولیه
Selection	population	parent1, parent2	انتخاب والدین
Crossover	parent1, parent2	child	تولید فرزند
Mutation	child	mutated child	ایجاد تنوع
Gemini generate_content	prompt	response.text	دستیار هوشمند

11-1. اصول خوانایی و ماژولار بودن

- تفکیک منطق حل از رابط کاربری.
- نام‌گذاری معنادار متغیرها و توابع.
- استفاده از کامنت‌های بخش‌بندی‌شده برای Objective، Constraints، Results و UI.
- ذخیره خروجی‌ها در پوشه outputs به جای پراکندگی فایل‌ها.
- جدا نگه داشتن API Key از سورس‌کد.
- قابل تنظیم کردن پارامترهای اصلی الگوریتم.

۱۲. آزمون و ارزیابی

ارزیابی پروژه در چند سطح انجام شده است: آزمون اجرای حل‌کننده LP، آزمون تولید خروجی Excel/نمودار، آزمون اجرای GA در چند نسل، آزمون تغییر Population Size و Generations، آزمون نمایش خروجی در Streamlit و آزمون ارتباط Chatbot با Gemini.

سناریو آزمون	انتظار	نتیجه مورد انتظار
اجرای LP با پارامتر پایه	مدل حل شود	Status=Optimal و خروجی تولید
تغییر ظرفیت /Paint/Body Assembly	جواب مدل نسبت به ظرفیت جدید محاسبه شود	سناریوی جدید تولید
اجرای GA	تولید جمعیت و نسل‌ها	Best Solution نمایش داده شود
Population=10/80	تعداد کروموزوم‌ها تغییر کند	اندازه جمعیت در خروجی تغییر کند
Generations=10/50	تعداد تکرارها تغییر کند	شماره نسل تا مقدار انتخابی ادامه یابد
Chatbot	پرسش تولیدی ارسال شود	پاسخ متنی Gemini نمایش داده شود
فایل خروجی وجود ندارد	خطا کنترل شود	پیام واضح به کاربر

12-1. معیارهای ارزیابی مطابق دستور پروژه

راهنمای پروژه، برای مدل‌سازی مهندسی صنایع تعریف دقیق مسئله و فرمول‌بندی را، برای LP صحت و خوانایی پیاده‌سازی را، برای GA طراحی کروموزوم و همگرایی را، برای GUI تجربه کاربری و نمودارها را و برای Chatbot سرعت پاسخ و درک خروجی‌های بهینه‌سازی را به عنوان شاخص‌های ارزیابی تعیین می‌کند. ❌cite

۱۳. محدودیت‌ها و پیشنهادهای توسعه

13-1. محدودیت‌های فعلی

• مدل GA فعلی یک زمان‌بندی ساده‌شده است و هنوز Job-Shop Scheduling چندماشینه کامل نیست.

- Fitness ترکیبی و آموزشی است و می‌تواند با معیارهای استانداردتر مانند makespan، lateness و machine utilization توسعه یابد.
- Shadow Price در MILP نباید بدون توجه به محدودیت‌های دوگان تفسیر اقتصادی مستقیم شود.
- اتصال Chatbot به خروجی آخرین اجرای LP/GA هنوز می‌تواند عمیق‌تر شود.
- در صورت انتشار پروژه، مدیریت Secret‌ها و حذف اطلاعات حساس از تصاویر/فایل‌های محیط توسعه ضروری است.

2-13. مسیرهای توسعه

- افزودن مدل Job-Shop با چند ایستگاه و چند ماشین.
- استفاده از Tournament Selection یا Elitism در GA.
- ذخیره تاریخچه Fitness هر نسل و رسم نمودار همگرایی.
- افزودن تحلیل سناریو و مقایسه چند اجرای GA.
- ساخت صفحه جداگانه برای Sensitivity Analysis.
- اتصال Chatbot به آخرین خروجی‌ها و تولید توضیح خودکار از نتایج.
- افزودن تست‌های واحد برای crossover، fitness و mutation.
- افزودن requirements.txt و README کامل برای تحویل.

۱۴. جمع‌بندی

پروژه Toyota Production Planning یک نمونه یکپارچه از کاربرد Python در تصمیم‌گیری مهندسی صنایع است که چهار لایه اصلی حل دقیق، بهینه‌سازی تکاملی، رابط وب و هوش مصنوعی را کنار یکدیگر قرار می‌دهد. مدل MILP بخش تصمیم‌گیری کمی را با متغیرهای صحیح، تابع هدف سود و قیود ظرفیت/تقاضا پوشش می‌دهد؛ الگوریتم ژنتیک نیز یک مسئله ترتیبی را با نمایش permutation، تابع Fitness، Crossover، Selection و Mutation حل می‌کند.

از دید مهندسی نرم‌افزار، Streamlit نقش لایه ارائه و orchestration را بر عهده دارد و خروجی‌های محاسباتی را به صورت جدول و نمودار در اختیار کاربر می‌گذارد. استفاده از Gemini نیز یک لایه تعاملی و تحلیلی به سامانه اضافه می‌کند. نتیجه نهایی، یک Decision Support System آموزشی و قابل توسعه

است که می‌تواند در آینده با مدل‌های زمان‌بندی صنعتی‌تر، تحلیل حساسیت کامل‌تر و Chatbot متصل به context محاسباتی ارتقا یابد.

از نظر انطباق با دستور پروژه، معماری چهارماژوله، مدل MILP، الگوریتم ژنتیک سفارشی، داشبورد Streamlit و دستیار هوشمند در ساختار سامانه لحاظ شده‌اند. گزارش حاضر نیز فرمول‌بندی ریاضی، منطق الگوریتم‌ها، ساختار نرم‌افزار، توابع کلیدی، تست‌ها و محدودیت‌ها را مستند می‌کند.

۱۵. پیوست: ساختار فایل‌ها و چک‌لیست تحویل

فایل/پوشه	کاربرد
app.py	رابط Streamlit و orchestration
lp_solver.py	مدل MILP و خروجی‌های LP
ga_scheduler.py	الگوریتم ژنتیک
production_data.csv	پارامترهای داده‌ای مدل LP
/outputs	فایل‌های Excel و نمودارهای تولیدشده
env.	نگهداری امن API Key؛ نباید منتشر شود
requirements.txt	وابستگی‌های Python
README.md	راهنمای نصب و اجرای پروژه

15-1. چک‌لیست نهایی

- اجرای app.py بدون خطا
- اجرای LP و تولید production_plan.xlsx
- نمایش نمودار تولید
- نمایش Slack و Shadow Price در محل مناسب
- اجرای GA با پارامترهای قابل تغییر
- نمایش Best Solution و Fitness

- اجرای Chatbot و کنترل خطاهای API
- حذف API Key از تصاویر و مخزن عمومی
- تکمیل requirements.txt و README.md
- افزودن لینک GitHub / Colab / AI Studio در بسته تحویل، مطابق دستور پروژه

منابع و مستندات پروژه

- (1) دستورالعمل پروژه پایانی درس برنامه‌نویسی پیشرفته، دانشکده مهندسی صنایع، دانشگاه صنعتی شریف.
- (2) کد منبع پروژه: app.py، lp_solver.py، ga_scheduler.py و فایل داده production_data.csv.
- (3) مستندات کتابخانه PuLP برای مدل‌سازی Linear/Mixed Integer Programming.
- (4) مستندات Streamlit برای توسعه رابط‌های تعاملی Python.
- (5) مستندات Google Gemini API برای تولید پاسخ‌های متنی و دستیار هوشمند.
- (6) این گزارش با رویکرد AI-Assisted Documentation تدوین و سپس بر اساس ساختار و کد پروژه بازبینی شده است.

لینک مخزن پروژه

مخزن GitHub:

https://github.com/aramshafieha/Toyota_Production_Planning

این مخزن شامل کدهای Python، فایل requirements.txt، داده‌ها، خروجی‌های محاسباتی، مستندات و فایل گزارش پروژه است.